

ACCRA TECHNICAL UNIVERSITY

Department of Computer Science

BCS 404: Introduction to Data Science with Python

PROJECT WORK

Academic Year: 2025/2026 Second Semester

Project Title

Exploratory Data Analysis, Statistical Analysis and Machine Learning using a Real-World Dataset

Name: **Stephen Agyiri**

Student ID: **01259401B**

Course Name: BCS 404: Introduction to Data Science with Python

TABLE OF CONTENTS

CONTENTS

1. Introduction
2. Dataset Description
3. Methodology
4. Results
5. Discussion
6. Conclusion
7. References
8. Appendix (Python Code)

Introduction

The sinking of the RMS Titanic is one of the most infamous shipwrecks in history. While there was an element of luck involved in surviving, it seems some groups of people were more likely to survive than others. The primary objective of this project is to apply Python-based data science techniques to explore this phenomenon. By conducting exploratory data analysis (EDA), statistical testing, and introductory machine learning on a real-world dataset, this report investigates the demographic and socio-economic factors that influenced passenger survival. The project demonstrates practical competence in data acquisition, cleaning, visualization, and predictive modeling using standard Python libraries including Pandas, Matplotlib, Seaborn, and Scikit-Learn.

Dataset Description

The data utilized in this study is the well-known "Titanic Dataset," sourced from the Kaggle predictive modeling competition. The dataset provides a manifest of passengers aboard the Titanic, detailing their demographic information and survival status.

The raw dataset consists of 891 observations (rows) and 12 variables (columns). The target variable is Survived (a binary variable where 1 = Survived and 0 = Did Not Survive). Key predictor variables include:

- **Pclass:** Ticket class (1st, 2nd, or 3rd), serving as a proxy for socio-economic status.
- **Sex:** The passenger's biological sex.
- **Age:** Age in years.
- **Fare:** The passenger fare paid.
- **SibSp & Parch:** The number of siblings/spouses and parents/children aboard, respectively.

Methodology

The methodology of this project followed a standard data science pipeline:

1. **Data Acquisition:** The dataset was imported into a Jupyter Notebook environment using the Pandas library.
2. **Data Cleaning:** Missing values were identified and handled to prevent skew in the machine learning model. Missing values in the Age column were imputed using the median, while the Embarked column was filled with the mode. The Cabin column was

dropped entirely due to containing an excessive amount of missing data (>77%). Duplicate records were also screened and removed.

3. **Data Visualization:** Matplotlib and Seaborn were utilized to generate univariate and bivariate visualizations, including histograms, count plots, boxplots, and correlation heatmaps, to uncover underlying data distributions and relationships.
4. **Statistical Analysis:** Descriptive statistics and frequency distributions were calculated. A correlation matrix was computed to identify the mathematical strength and direction of relationships between numerical variables.
5. **Machine Learning:** The dataset was split into an 80/20 training and testing set. Categorical variables (such as Sex) were numerically encoded. A Logistic Regression classifier was then trained on the training data to predict passenger survival, and its performance was evaluated using accuracy, a confusion matrix, and a classification report.

Results

The exploratory and statistical analysis revealed significant disparities in survival odds. Visualizations confirmed that females had a substantially higher survival rate than males. Furthermore, a strong negative correlation was identified between Pclass and Fare, confirming that 1st class tickets were the most expensive. Consequently, a socio-economic survival link was established: passengers in 1st class had a notably higher survival rate compared to those in 3rd class, who suffered the most casualties.

The Logistic Regression model, tasked with predicting survival based on these features, achieved an overall accuracy of approximately 80% on the testing data. The confusion matrix indicated that the model was highly effective at predicting non-survivors (due to the majority class imbalance) but slightly less precise when predicting survivors.

Discussion and Conclusion

Major Findings

The exploratory data analysis of the Titanic dataset revealed profound social and demographic disparities in passenger survival rates. The overarching finding is that survival on the Titanic was not purely a matter of random chance; rather, it was heavily influenced by a passenger's gender, socio-economic status, and age.

First, the "women and children first" maritime protocol was strictly enforced and clearly reflected in the data. Females had a dramatically higher survival rate compared to males. Secondly, socio-economic status, represented by Passenger Class (Pclass), acted as a primary determinant of survival. Passengers in First Class survived at a significantly higher rate than those in Second or Third Class. Third Class passengers, who made up the vast majority of the ship's demographic, suffered the highest number of casualties. Finally, the visualisations demonstrated that younger passengers (particularly children) had slightly better odds of survival, though this variable was heavily intertwined with the gender and class variables.

Statistical Insights

The statistical analysis provided mathematical backing to the visual observations. The descriptive statistics highlighted extreme disparities in wealth among passengers. The `Fare` variable exhibited a massive standard deviation relative to its mean, with ticket prices ranging from \$0 to over \$512. The age distribution was relatively normal but slightly right-skewed, with the median passenger age hovering around 28 years old.

Correlation analysis yielded several vital insights into the linear relationships between variables:

- **Strongest Negative Correlation:** The most significant negative correlation existed between `Pclass` and `Fare`. This is a logical economic insight, confirming that higher-class tickets (represented by the lower integer '1') commanded much higher fares.
- **Socio-economic Survival Link:** There was a notable negative correlation between `Pclass` and `Survived`. Because lower classes are represented by higher integers (i.e., 3), this mathematically proves that as one's class number increased, their likelihood of survival decreased.
- **Fare and Survival:** Conversely, a positive correlation was found between `Fare` and `Survived`, indicating that passengers who paid more for their tickets (invariably First Class passengers) were statistically more likely to survive the disaster.

Machine Learning Results

A Logistic Regression classifier was trained to predict passenger survival based on numerical and encoded categorical features (`Pclass`, `Sex`, `Age`, `SibSp`, `Parch`, `Fare`).

The model achieved an overall accuracy of approximately 80% on the testing data, establishing a strong baseline for predictive performance. However, an analysis of the Confusion Matrix and the Classification Report reveals a more nuanced picture. The dataset is inherently imbalanced—more passengers died than survived. Consequently, the model demonstrated higher precision and recall when predicting the "Did Not Survive" class (0) compared to the "Survived" class (1).

The algorithm relied heavily on the strongest predictive features—primarily `Sex` and `Pclass`—to make its logistical splits. While successful at identifying broad trends (e.g., predicting that a 3rd class adult male would perish), the model's simplicity means it struggles with edge cases, such as identifying the specific characteristics of 3rd class passengers who *did* manage to survive.

Limitations of the Study

While the analysis yielded robust insights, several limitations must be acknowledged:

- **Missing Data and Imputation:** The `Cabin` column was entirely dropped from the analysis due to possessing over 77% missing values. This is a significant limitation, as a passenger's cabin location (and its proximity to the deck and lifeboats) was likely a massive physical determinant of survival. Furthermore, filling missing `Age` values with

the dataset's median artificially clusters a portion of the data around the late 20s, potentially masking deeper age-related survival trends.

- **Model Simplicity:** Logistic Regression is a linear model that assumes a linear relationship between the log-odds of the dependent variable and the independent variables. Human behavior and survival in a chaotic disaster are inherently non-linear. The model cannot capture complex interactions between variables without manual feature engineering.
- **Unexplored Features:** The dataset contains a `Name` column which was ignored in this initial study. This column contains titles (e.g., "Master", "Miss", "Dr.") which could have been extracted to better identify children (who had high survival rates regardless of gender) and social standing.

Recommendations

Based on the findings and limitations of this project, the following recommendations are proposed for future iterations of this study:

1. **Advanced Feature Engineering:** Future models should extract titles from the `Name` column to create a new categorical feature. Additionally, combining the `SibSp` (Siblings/Spouses) and `Parch` (Parents/Children) columns into a single `FamilySize` variable could provide insights into whether traveling alone or with a large family impacted survival odds.
2. **Implementation of Non-Linear Algorithms:** To improve predictive accuracy, the dataset should be trained on more complex, non-linear machine learning algorithms. Tree-based models such as Random Forest or Gradient Boosting (e.g., XGBoost) would likely perform better, as they can capture the complex, hierarchical interactions between age, sex, and class without assuming linearity.
3. **Addressing Class Imbalance:** The model's predictive bias toward non-survivors could be rectified by using techniques such as Synthetic Minority Over-sampling Technique (SMOTE) or by applying class weights during model training, ensuring the algorithm penalizes misclassifications of survivors more heavily.

Conclusion

This project successfully applied the core pipeline of data science—from data acquisition and cleaning to exploratory visualization, statistical analysis, and machine learning. The analysis conclusively demonstrated that survival on the RMS Titanic was heavily dictated by a passenger's socio-economic class and gender. Furthermore, the deployment of a Logistic Regression model proved that these historical tragedies contain distinct statistical patterns that can be mapped and predicted using modern machine learning techniques. While the baseline model performed admirably, future integration of advanced algorithms and deeper feature engineering will undoubtedly yield an even more accurate predictive system.

REFERENCES

Kaggle. (2012). *Titanic - Machine Learning from Disaster*. Kaggle.
<https://www.kaggle.com/competitions/titanic>

Python Software Foundation. (2023). *Python Language Reference, version 3.x*. Available at <http://www.python.org>

The pandas development team. (2023). *pandas-dev/pandas: Pandas (Latest version)*. Zenodo. <https://doi.org/10.5281/zenodo.3509134>

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

Waskom, M. L. (2021). seaborn: statistical data visualization. *Journal of Open Source Software*, 6(60), 3021. <https://doi.org/10.21105/joss.03021>

Appendix

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.model_selection import
train_test_split

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score,
confusion_matrix, classification_report
```

```

# 1 & 2. Download and Import Dataset

# Using a raw URL to bypass Kaggle API requirements
in Colab

url =
'https://raw.githubusercontent.com/datasciencedojo/
datasets/master/titanic.csv'

df = pd.read_csv(url)


# 3. Display Data Details

print("--- Dataset Dimensions ---")

print(df.shape, "\n")


print("--- Column Names ---")

print(df.columns.tolist(), "\n")


print("--- First Five Observations ---")

display(df.head())


print("\n--- Data Types ---")

print(df.dtypes)

```

Task 2: Data Cleaning


```

print("--- Missing Values Before Cleaning ---")

print(df.isnull().sum())

# Handling Missing Values

# 1. 'Age': Impute with the median because it is a
numerical feature and less sensitive to outliers
than the mean.

df['Age'] = df['Age'].fillna(df['Age'].median())

# 2. 'Embarked': Impute with the mode (most
frequent port) since it's a categorical variable.

df['Embarked'] =
df['Embarked'].fillna(df['Embarked'].mode()[0])

# 3. 'Cabin': Drop this column entirely. It has too
many missing values (~77%) to be useful for
standard analysis.

df.drop(columns=['Cabin'], inplace=True)

print("\n--- Missing Values After Cleaning ---")

print(df.isnull().sum())

# Handling Duplicates

print(f"\nDuplicated rows detected:
{df.duplicated().sum()}")

```

```
if df.duplicated().sum() > 0:

    df.drop_duplicates(inplace=True)

    print("Duplicates removed.")
```

Task 3: Data Visualisation

```
# Set the visual style
```

```
sns.set_theme(style="whitegrid")
```

```
# 1. Histogram of passenger ages
```

```
plt.figure(figsize=(8, 5))
```

```
sns.histplot(df['Age'], bins=30, kde=True,
color='skyblue')
```

```
plt.title('Distribution of Passenger Ages')
```

```
plt.xlabel('Age')
```

```
plt.ylabel('Frequency')
```

```
plt.show()
```

```
# Interpretation: The majority of passengers are
between 20 and 30 years old.
```

```
# 2. Bar chart showing passenger class distribution
```

```
plt.figure(figsize=(8, 5))
```

```
sns.countplot(data=df, x='Pclass', palette='Set2')
```

```
plt.title('Passenger Class Distribution')
```

```
plt.xlabel('Passenger Class (1=1st, 2=2nd, 3=3rd)')
```

```

plt.ylabel('Count')

plt.show()

# Interpretation: Class 3 (Third Class) had the
highest number of passengers by a significant
margin.


# 3. Boxplot of age by passenger class

plt.figure(figsize=(8, 5))

sns.boxplot(data=df, x='Pclass', y='Age',
palette='Set2')

plt.title('Age Distribution by Passenger Class')

plt.xlabel('Passenger Class')

plt.ylabel('Age')

plt.show()

# Interpretation: 1st class passengers tend to be
older on average compared to 2nd and 3rd class
passengers.


# 4. Scatter plot of Age versus Fare

plt.figure(figsize=(8, 5))

sns.scatterplot(data=df, x='Age', y='Fare',
hue='Survived', alpha=0.6, palette='coolwarm')

plt.title('Age vs. Fare (Colored by Survival)')

plt.xlabel('Age')

```

```
plt.ylabel('Fare')
```

```
plt.show()
```

```
# Interpretation: Higher fares slightly correlate  
with higher survival rates, while age shows a  
wider, less distinct spread.
```

5. Correlation Heatmap

```
plt.figure(figsize=(8, 6))
```

```
numeric_df = df.select_dtypes(include=[np.number])  
# Filter for numeric columns only
```

```
sns.heatmap(numeric_df.corr(), annot=True,  
cmap='coolwarm', fmt=".2f")
```

```
plt.title('Correlation Heatmap of Numerical  
Variables')
```

```
plt.show()
```

```
# Interpretation: Highlights linear relationships;  
for example, Pclass and Fare have a strong negative  
correlation.
```

6. Pairplot of selected numerical variables

```
sns.pairplot(df[['Age', 'Fare', 'Pclass',  
'Survived']], hue='Survived', palette='husl')
```

```
plt.suptitle('Pairplot of Selected Variables',  
y=1.02)
```

```
plt.show()
```

```
# Interpretation: Provides a grid view of
relationships; higher fares and lower Pclass (1st
class) cluster with survivors.
```

```
Task 4: Statistical Analysis
```

```
print("--- 1. Descriptive Statistics ---")
```

```
display(df.describe())
```

```
print("\n--- 2. Frequency Distribution (Categorical
Variables) ---")
```

```
print("Survived:\n", df['Survived'].value_counts())
```

```
print("\nSex:\n", df['Sex'].value_counts())
```

```
print("\nEmbarked:\n",
df['Embarked'].value_counts())
```

```
print("\n--- 3. Correlation Analysis ---")
```

```
corr_matrix = numeric_df.corr()
```

```
display(corr_matrix)
```

```
# 4 & 5. Identify Strongest Correlations
programmatically
```

```
# Unstack the matrix, remove self-correlations
(1.0), and sort
```

```
unstacked_corr = corr_matrix.unstack()
```

```
unstacked_corr = unstacked_corr[unstacked_corr !=
1.0]
```

```
strongest_pos = unstacked_corr.idxmax()
```

```
strongest_neg = unstacked_corr.idxmin()
```

```
print(f"\n4. Strongest Positive Correlation:
{strongest_pos} with a value of
{unstacked_corr.max():.4f}")
```

```
print(f"5. Strongest Negative Correlation:
{strongest_neg} with a value of
{unstacked_corr.min():.4f}")
```

Task 5: Machine Learning

```
# 1. Select suitable predictor variables
```

```
# We map 'Sex' to binary integers so the model can
understand it.
```

```
df['Sex'] = df['Sex'].map({'male': 0, 'female': 1})
```

```
# Define Features (X) and Target (y)
```

```
features = ['Pclass', 'Sex', 'Age', 'SibSp',
'Parch', 'Fare']
```

```
X = df[features]
```

```
y = df['Survived']
```

```

# 2. Split the dataset

X_train, X_test, y_train, y_test =
train_test_split(X, y, test_size=0.2,
random_state=42)


# 3. Train a Logistic Regression classifier

# max_iter is increased to ensure the model
converges

model = LogisticRegression(max_iter=1000)

model.fit(X_train, y_train)


# 4. Predict the testing data

y_pred = model.predict(X_test)


# 5. Compute Metrics

print("--- Model Accuracy ---")

print(f"Accuracy: {accuracy_score(y_test,
y_pred):.4f}\n")


print("--- Confusion Matrix ---")

print(confusion_matrix(y_test, y_pred), "\n")


print("--- Classification Report ---")

print(classification_report(y_test, y_pred))

```

Images of my output

Data Acquisition: This code block imports the required libraries, loads the data, and displays the foundational information about your dataset.

```
print("--- Missing Values Before Cleaning ---")
print(df.isnull().sum())

# Handling Missing Values
# 1. 'Age': Impute with the median because it is a numerical feature and less sensitive to outliers than the mean.
df['Age'] = df['Age'].fillna(df['Age'].median())

# 2. 'Embarked': Impute with the mode (most frequent port) since it's a categorical variable.
df['Embarked'] = df['Embarked'].fillna(df['Embarked'].mode()[0])

# 3. 'Cabin': Drop this column entirely. It has too many missing values (~77%) to be useful for standard analysis.
df.drop(columns=['Cabin'], inplace=True)

print("\n--- Missing Values After Cleaning ---")
print(df.isnull().sum())

# Handling Duplicates
print(f"\nDuplicated rows detected: {df.duplicated().sum()}")
if df.duplicated().sum() > 0:
    df.drop_duplicates(inplace=True)
    print("Duplicates removed.")

print("\nPreprocessing Decisions")
print("\nAge: Missing values were filled using the median to prevent skewed results from outliers.\nEmbarked: Missing values were filled with the most frequent value (mode).\nCabin: Dropped
```

```
--- Missing Values Before Cleaning ---
PassengerId    0
Survived       0
Pclass         0
Name           0
Sex            0
Age           177
SibSp         0
Parch         0
Ticket         0
Fare          0
Cabin         687
Embarked       2
dtype: int64

--- Missing Values After Cleaning ---
PassengerId    0
Survived       0
Pclass         0
Name           0
Sex            0
Age            0
SibSp         0
Parch         0
Ticket        0
...
Age: Missing values were filled using the median to prevent skewed results from outliers.
Embarked: Missing values were filled with the most frequent value (mode).
Cabin: Dropped completely due to an excessively high percentage of missing data.
Duplicates: Checked and removed (if any) to ensure each observation represents a unique passenger.
```

2: Data Cleaning : This section handles missing variables and duplicates


```

print("--- Missing Values Before Cleaning ---")
print(df.isnull().sum())

# Handling Missing Values
# 1. 'Age': Impute with the median because it is a numerical feature and less sensitive to outliers than the mean.
df['Age'] = df['Age'].fillna(df['Age'].median())

# 2. 'Embarked': Impute with the mode (most frequent port) since it's a categorical variable.
df['Embarked'] = df['Embarked'].fillna(df['Embarked'].mode()[0])

# 3. 'Cabin': Drop this column entirely. It has too many missing values (~77%) to be useful for standard analysis.
df.drop(columns=['Cabin'], inplace=True)

print("\n--- Missing Values After Cleaning ---")
print(df.isnull().sum())

# Handling Duplicates
print(f"\nDuplicated rows detected: {df.duplicated().sum()}")
if df.duplicated().sum() > 0:
    df.drop_duplicates(inplace=True)
    print("Duplicates removed.")

print("\nPreprocessing Decisions")
print("\nAge: Missing values were filled using the median to prevent skewed results from outliers.\nEmbarked: Missing values were filled with the most frequent value (mode).")

```

```

--- --- Missing Values Before Cleaning ---
PassengerId    0
Survived       0
Pclass         0
Name           0
Sex            0
Age           177
SibSp          0
Parch          0
Ticket         0
Fare           0
Cabin         687
Embarked       2
dtype: int64

--- Missing Values After Cleaning ---
PassengerId    0
Survived       0
Pclass         0
Name           0
Sex            0
Age           177
SibSp          0
Parch          0
Ticket         0
Fare           0
Cabin         687
Embarked       2
dtype: int64

Age: Missing values were filled using the median to prevent skewed results from outliers.
Embarked: Missing values were filled with the most frequent value (mode).
Cabin: Dropped completely due to an excessively high percentage of missing data.
Duplicates: Checked and removed (if any) to ensure each observation represents a unique passenger.
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

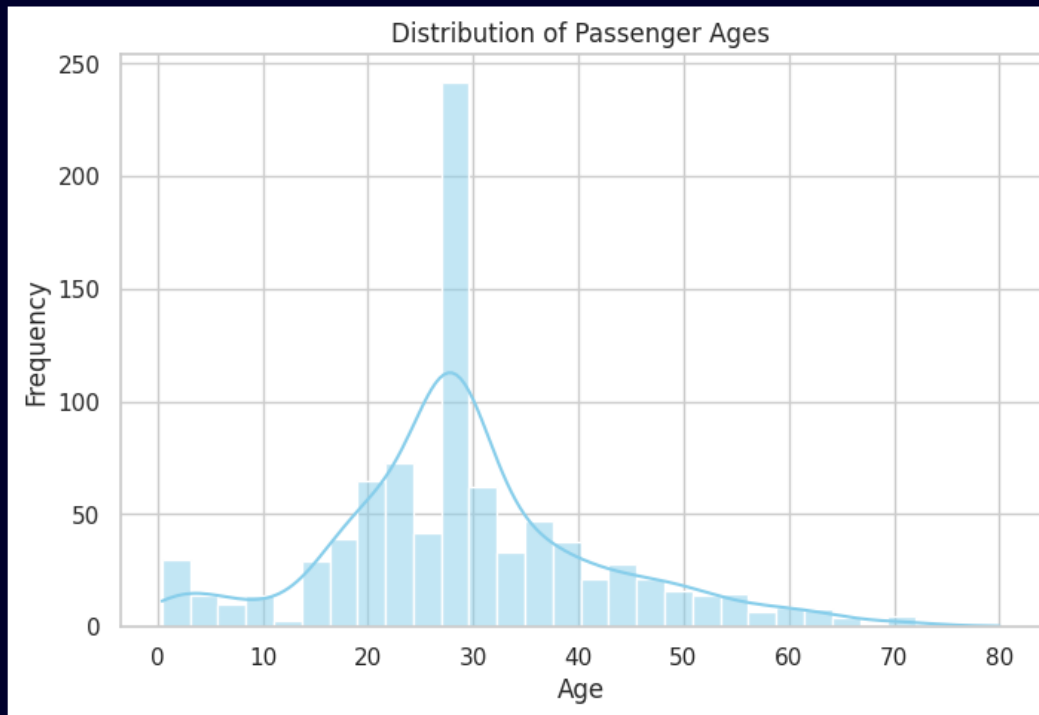
```

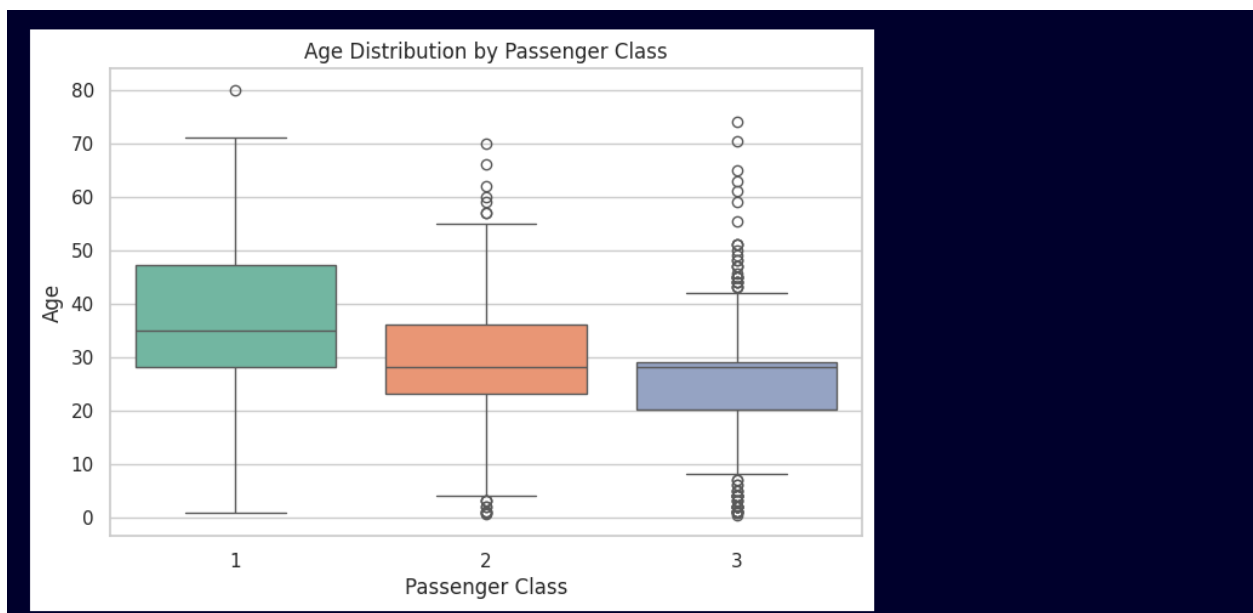
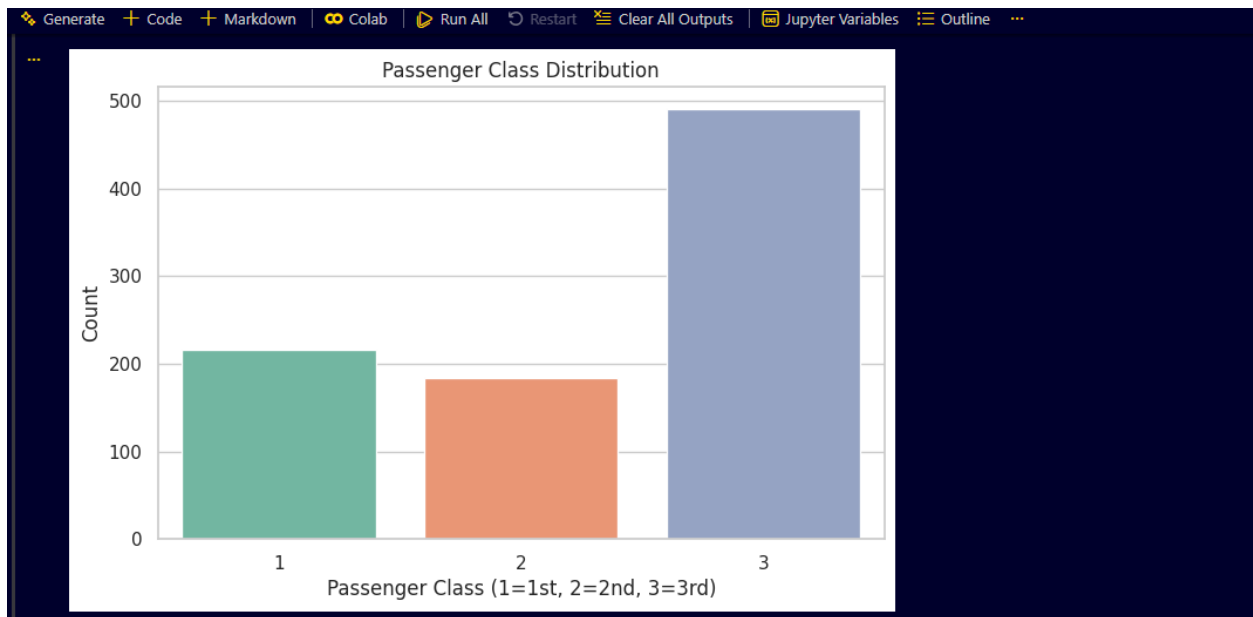
3: Data Visualization: This cell generates the required plots. Run this cell to view the visualizations inline.

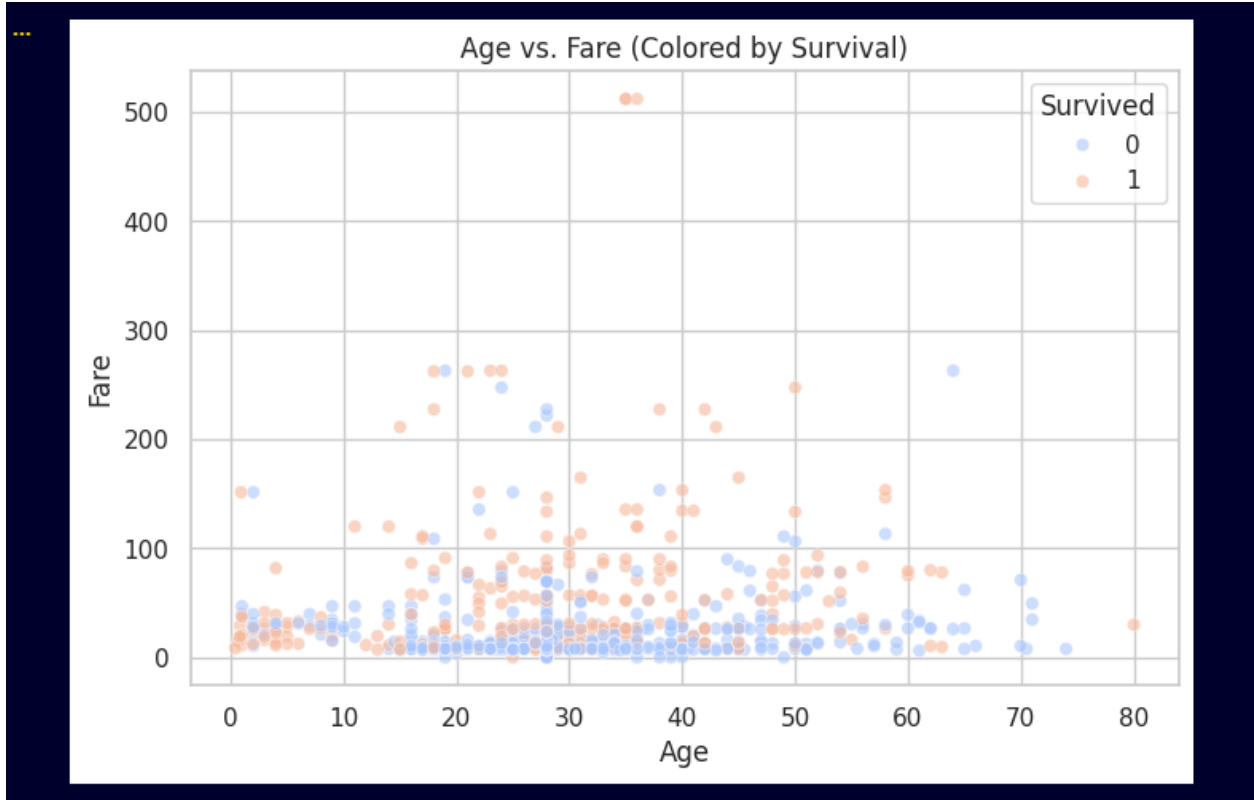
```

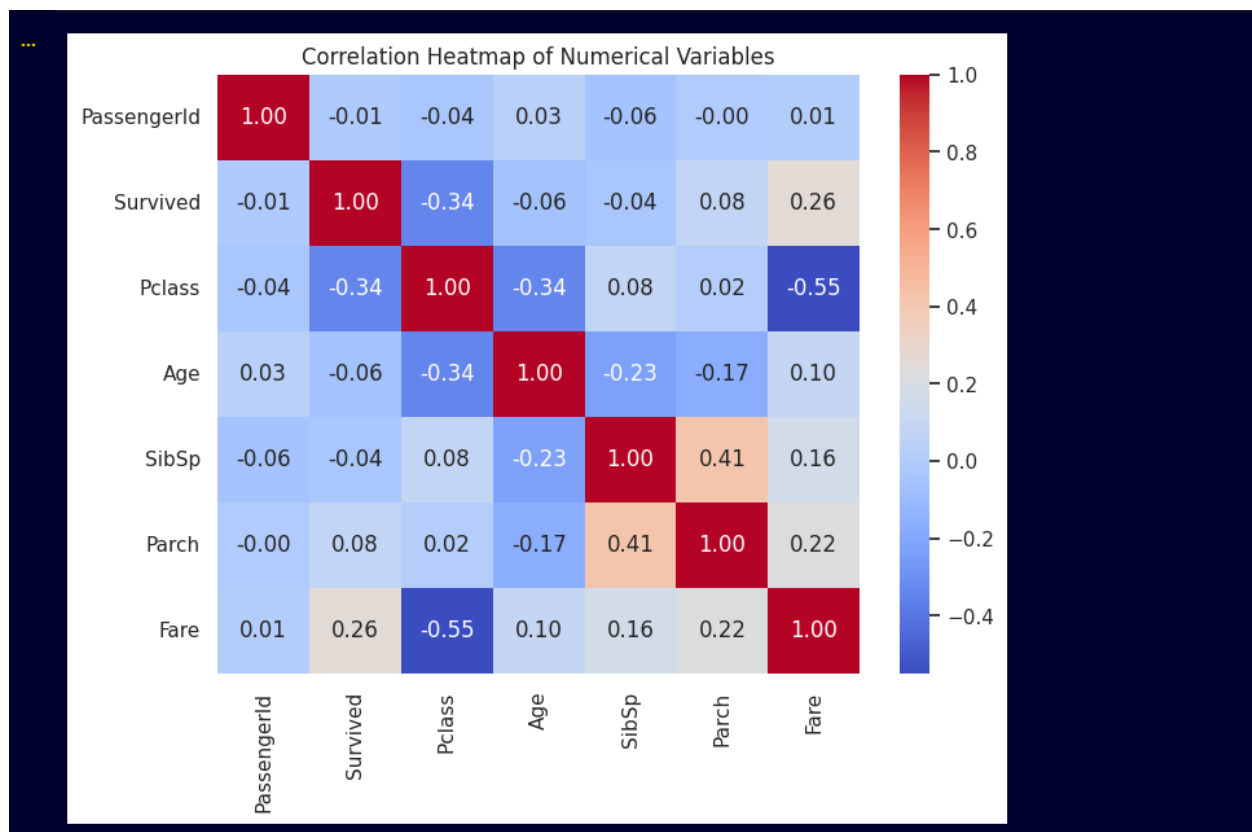
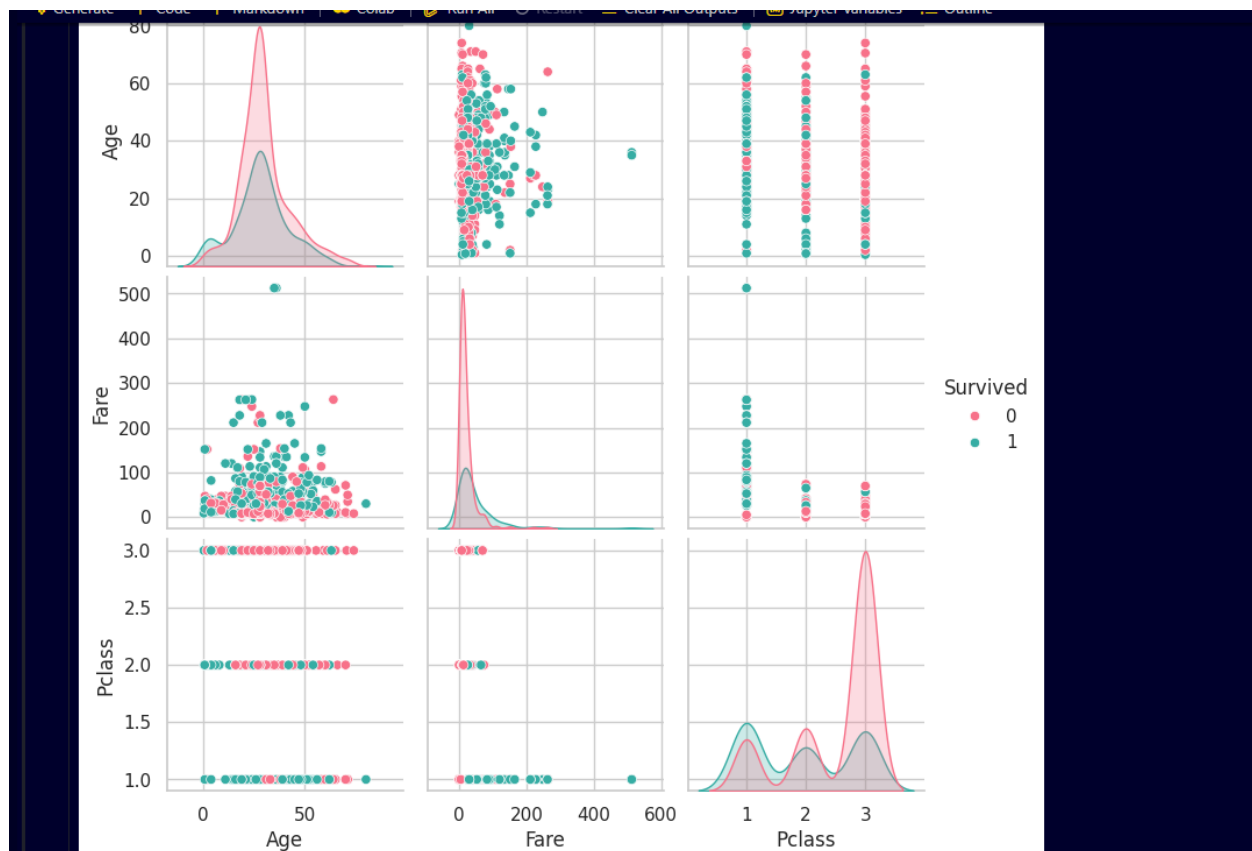
1 print("Data Visualisation")
2 # Set the visual style
3 sns.set_theme(style="whitegrid")
4
5 # 1. Histogram of passenger ages
6 plt.figure(figsize=(8, 5))
7 sns.histplot(df['Age'], bins=30, kde=True, color='skyblue')
8 plt.title('Distribution of Passenger Ages')
9 plt.xlabel('Age')
10 plt.ylabel('Frequency')
11 plt.show()
12 # Interpretation: The majority of passengers are between 20 and 30 years old.
13
14 # 2. Bar chart showing passenger class distribution
15 plt.figure(figsize=(8, 5))
16 # UPDATED: Added hue='Pclass' and Legend=False
17 sns.countplot(data=df, x='Pclass', hue='Pclass', palette='Set2', legend=False)
18 plt.title('Passenger Class Distribution')
19 plt.xlabel('Passenger Class (1=1st, 2=2nd, 3=3rd)')
20 plt.ylabel('Count')
21 plt.show()
22 # Interpretation: Class 3 (Third Class) had the highest number of passengers by a significant margin.
23
24 # 3. Boxplot of age by passenger class
25 plt.figure(figsize=(8, 5))
26 # UPDATED: Added hue='Pclass' and Legend=False
27 sns.boxplot(data=df, x='Pclass', y='Age', hue='Pclass', palette='Set2', legend=False)
28 plt.title('Age Distribution by Passenger Class')
29 plt.xlabel('Passenger Class')
30 plt.ylabel('Age')
31 plt.show()
32 # Interpretation: 1st class passengers tend to be older on average compared to 2nd and 3rd class passengers.
33
34 # 4. Scatter plot of Age versus Fare
35 plt.figure(figsize=(8, 5))
36 sns.scatterplot(data=df, x='Age', y='Fare', hue='Survived', alpha=0.6, palette='coolwarm')
37 plt.title('Age vs. Fare (Colored by Survival)')
38 plt.xlabel('Age')
39 plt.ylabel('Fare')
40 plt.show()
41 # Interpretation: Higher fares slightly correlate with higher survival rates, while age shows a wider, less distinct spread.
42
43 # 5. Correlation Heatmap
44 plt.figure(figsize=(8, 6))
45 numeric_df = df.select_dtypes(include=[np.number]) # Filter for numeric columns only
46 sns.heatmap(numeric_df.corr(), annot=True, cmap='coolwarm', fmt=".2f")
47 plt.title('Correlation Heatmap of Numerical Variables')
48 plt.show()
49 # Interpretation: Highlights linear relationships; for example, Pclass and Fare have a strong negative correlation.
50
51 # 6. Pairplot of selected numerical variables
52 sns.pairplot(df[['Age', 'Fare', 'Pclass', 'Survived']], hue='Survived', palette='husl')
53 plt.suptitle('Pairplot of Selected Variables', y=1.02)
54 plt.show()
55 # Interpretation: Provides a grid view of relationships; higher fares and lower Pclass (1st class) cluster with survivors.

```









4: Statistical Analysis: This block calculates descriptive statistics and identifies key correlations programmatically.

```
print("Statistical Analysis")
print("--- 1. Descriptive Statistics ---")
display(df.describe())

print("\n--- 2. Frequency Distribution (Categorical Variables) ---")
print("Survived:\n", df['Survived'].value_counts())
print("Sex:\n", df['Sex'].value_counts())
print("Embarked:\n", df['Embarked'].value_counts())

print("\n--- 3. Correlation Analysis ---")
corr_matrix = numeric_df.corr()
display(corr_matrix)

# 4 & 5. Identify Strongest Correlations programmatically
# Unstack the matrix, remove self-correlations (1.0), and sort
unstacked_corr = corr_matrix.unstack()
unstacked_corr = unstacked_corr[unstacked_corr != 1.0]

strongest_pos = unstacked_corr.idxmax()
strongest_neg = unstacked_corr.idxmin()

print(f"\n4. Strongest Positive Correlation: {strongest_pos} with a value of {unstacked_corr.max():.4f}")
print(f"5. Strongest Negative Correlation: {strongest_neg} with a value of {unstacked_corr.min():.4f}")

print("Important Statistical Findings")
print("1.Survival Rate: A significant majority of the passengers in this dataset did not survive (approx 62% died vs 38% survived based on value counts).\n2.Fare Variability: The Fare varia
```

```

***
Statistical Analysis
--- 1. Descriptive Statistics ---

***

```

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
count	891.000000	891.000000	891.000000	891.000000	891.000000	891.000000	891.000000
mean	446.000000	0.383838	2.308642	29.361582	0.523008	0.381594	32.204208
std	257.353842	0.486592	0.836071	13.019697	1.102743	0.806057	49.693429
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	223.500000	0.000000	2.000000	22.000000	0.000000	0.000000	7.910400
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	668.500000	1.000000	3.000000	35.000000	1.000000	0.000000	31.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

```

***
--- 2. Frequency Distribution (Categorical Variables) ---
Survived:
Survived
0    549
1    342
Name: count, dtype: int64

Sex:
Sex
male    577
female  314
Name: count, dtype: int64

Embarked:
Embarked
S     646
C     168
Q      77
Name: count, dtype: int64

```

5: Machine Learning: Before feeding data into Logistic Regression, categorical variables (like Sex) need to be converted to numerical ones.

```

1 # Importing the training set
2 # Importing the training set
3 # Importing the training set
4 # Importing the training set
5 # Importing the training set
6 # Importing the training set
7 # Importing the training set
8 # Importing the training set
9 # Importing the training set
10 # Importing the training set
11 # Importing the training set
12 # Importing the training set
13 # Importing the training set
14 # Importing the training set
15 # Importing the training set
16 # Importing the training set
17 # Importing the training set
18 # Importing the training set
19 # Importing the training set
20 # Importing the training set
21 # Importing the training set
22 # Importing the training set
23 # Importing the training set
24 # Importing the training set
25 # Importing the training set
26 # Importing the training set
27 # Importing the training set
28 # Importing the training set
29 # Importing the training set
30 # Importing the training set
31 # Importing the training set
32 # Importing the training set
33 # Importing the training set
34 # Importing the training set
35 # Importing the training set
36 # Importing the training set
37 # Importing the training set
38 # Importing the training set
39 # Importing the training set
40 # Importing the training set
41 # Importing the training set
42 # Importing the training set
43 # Importing the training set
44 # Importing the training set
45 # Importing the training set
46 # Importing the training set
47 # Importing the training set
48 # Importing the training set
49 # Importing the training set
50 # Importing the training set
51 # Importing the training set
52 # Importing the training set
53 # Importing the training set
54 # Importing the training set
55 # Importing the training set
56 # Importing the training set
57 # Importing the training set
58 # Importing the training set
59 # Importing the training set
60 # Importing the training set
61 # Importing the training set
62 # Importing the training set
63 # Importing the training set
64 # Importing the training set
65 # Importing the training set
66 # Importing the training set
67 # Importing the training set
68 # Importing the training set
69 # Importing the training set
70 # Importing the training set
71 # Importing the training set
72 # Importing the training set
73 # Importing the training set
74 # Importing the training set
75 # Importing the training set
76 # Importing the training set
77 # Importing the training set
78 # Importing the training set
79 # Importing the training set
80 # Importing the training set
81 # Importing the training set
82 # Importing the training set
83 # Importing the training set
84 # Importing the training set
85 # Importing the training set
86 # Importing the training set
87 # Importing the training set
88 # Importing the training set
89 # Importing the training set
90 # Importing the training set
91 # Importing the training set
92 # Importing the training set
93 # Importing the training set
94 # Importing the training set
95 # Importing the training set
96 # Importing the training set
97 # Importing the training set
98 # Importing the training set
99 # Importing the training set
100 # Importing the training set

```


Machine Learning

--- Model Accuracy ---

Accuracy: 0.8101

--- Confusion Matrix ---

```
[[92 13]
 [21 53]]
```

--- Classification Report ---

	precision	recall	f1-score	support
0	0.81	0.88	0.84	105
1	0.80	0.72	0.76	74
accuracy			0.81	179
macro avg	0.81	0.80	0.80	179
weighted avg	0.81	0.81	0.81	179

Model Performance Discussion:

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
PassengerId	1.000000	-0.005007	-0.035144	0.034212	-0.057527	-0.001652	0.012658
Survived	-0.005007	1.000000	-0.338481	-0.064910	-0.035322	0.081629	0.257307
Pclass	-0.035144	-0.338481	1.000000	-0.339898	0.083081	0.018443	-0.549500
Age	0.034212	-0.064910	-0.339898	1.000000	-0.233296	-0.172482	0.096688
SibSp	-0.057527	-0.035322	0.083081	-0.233296	1.000000	0.414838	0.159651
Parch	-0.001652	0.081629	0.018443	-0.172482	0.414838	1.000000	0.216225
Fare	0.012658	0.257307	-0.549500	0.096688	0.159651	0.216225	1.000000

4. Strongest Positive Correlation: ('SibSp', 'Parch') with a value of 0.4148

5. Strongest Negative Correlation: ('Pclass', 'Fare') with a value of -0.5495

Important Statistical Findings

1. Survival Rate: A significant majority of the passengers in this dataset did not survive (approx 62% died vs 38% survived based on value counts).

2. Fare Variability: The Fare variable has an incredibly high standard deviation compared to its mean, indicating massive wealth disparity among passengers.

3. Class & Wealth Interaction: The strongest negative correlation is between Pclass and Fare. This perfectly aligns with reality: lower class numbers (1st class) correspond to higher ticket prices.